



Universidad de Caldas
Facultad de Ingeniería
Análisis y Diseño de Algoritmos 2026-1

Manual de Usuario **K-QGMIP**

Extensión a k -Particiones de GeoMIP y QNodes

Integrantes del equipo:

Juan Sebastián García Hincapíe · ID: SebastianG210

Universidad de Caldas, Manizales, Colombia

Versión 1.0

Junio 2026

Repositorio: github.com/SebastianG210/proyectoAlgoritmos

Índice

1 Introducción y Visión General

1.1 ¿Qué es K-QGMIP?

K-QGMIP es una herramienta de software desarrollada en Python para analizar sistemas de variables binarias con dinámicas estocásticas. Su objetivo principal es encontrar la **partición mínima de información** (*k-MIP*) de un sistema: la forma de dividir las variables en k grupos de modo que se pierda la menor cantidad posible de información al tratar cada grupo de forma independiente.

Nota

Este software está diseñado para investigadores e ingenieros interesados en la **Teoría de la Información Integrada (IIT)**: un marco matemático que estudia cómo los sistemas integran información y que está relacionado con el estudio de la conciencia en sistemas físicos.

1.2 ¿Para qué sirve?

El software responde a la pregunta: *¿cuál es la mejor manera de dividir las n variables de un sistema en exactamente k grupos, de forma que la pérdida de información sea mínima?*

Aplicaciones típicas:

- Análisis de redes neuronales artificiales o biológicas.
- Estudio de dependencias entre variables en sistemas complejos.
- Comparación de múltiples estrategias de particionamiento ($k = 2$ hasta $k = 5$).
- Exploración de cómo aumentar k reduce la pérdida de información.

1.3 Estrategias Disponibles

K-QGMIP implementa cuatro estrategias (dos bases y dos extensiones):

Cuadro 1: Estrategias de particionamiento disponibles.

Estrategia	k soportado	Descripción breve
GeoMIP	$k = 2$	Búsqueda geométrica en hipercubo
KGeoMIP	$k = 2 \dots 5$	Extensión de GeoMIP a k grupos
QNodes	$k = 2$	Heurística MAO con oráculo submodular
KQNodes	$k = 2 \dots 5$	Extensión de QNodes a k grupos

1.4 Resultado Principal: el Dashboard

Al terminar el benchmark, el software genera un archivo **dashboard.html** que puede abrirse en cualquier navegador web. Contiene:

1. **Tiempos de ejecución** por instancia y estrategia (gráfico de líneas interactivo).

2. **Correlación de pérdidas** entre estrategias pares (diagramas de dispersión).
3. **Distribución estadística** de tiempos (boxplots).

1.5 Convenciones de Este Manual

- `texto` as indica comandos de terminal o nombres de archivos/carpetas.
- **Negrita** marca conceptos importantes la primera vez que aparecen.
- Las cajas de colores señalan información especial:

Nota

Información útil o contexto adicional.

Aviso

Advertencia: algo podría salir mal si no se sigue la indicación.

Importante

Acción que puede dañar datos o generar resultados incorrectos.

Consejo

Sugerencia para trabajar de forma más eficiente.

2 Requisitos del Sistema

2.1 Sistema Operativo

Cuadro 2: Compatibilidad de sistemas operativos.

Sistema	Versión mínima	Estado
Windows	10 / 11 (64-bit)	Verificado
Linux	Ubuntu 20.04+ o equivalente	Compatible
macOS	12 Monterey +	Compatible (sin síntesis de voz)

2.2 Python

Se requiere **Python 3.10 o superior** (se recomienda 3.12).

```
python --version
# Se espera: Python 3.10.x o superior
```

Listing 1: Verificar versión de Python.

Aviso

Python 3.9 o inferior no es compatible. Algunas características de tipado estático (x Yl syntax) requieren Python 3.10+.

2.3 Hardware Recomendado

Cuadro 3: Recursos de hardware recomendados según el tamaño de la red analizada.

Caso de uso	CPU	RAM	Disco
Redes pequeñas ($n \leq 10$)	2 núcleos	4 GB	500 MB
Redes medianas ($n \leq 15$)	4 núcleos	8 GB	1 GB
Redes grandes ($n \geq 20$)	8 núcleos	16 GB	5 GB

Importante

Las redes con $n \geq 20$ y $k = 5$ en modo exhaustivo requieren hasta 16 GB de RAM. Si el sistema no dispone de suficiente memoria, el proceso puede ser terminado por el sistema operativo.

2.4 Dependencias de Python

Las siguientes bibliotecas se instalan automáticamente al ejecutar la instalación:

Cuadro 4: Dependencias requeridas.

Biblioteca	Versión mínima	Propósito
numpy	2.0.0	Operaciones matriciales, hipercubos
plotly	6.0.0	Dashboard interactivo HTML
pandas	2.0.0	Lectura y escritura de CSV
colorama	0.4.6	Salida de terminal con colores
pyinstrument	5.0.0	Profiling de rendimiento
pyttsx3	2.99	Síntesis de voz (Windows)

Nota

El módulo `pyttsx3` es opcional en Linux/macOS. Si no está disponible, el software funciona normalmente pero sin el anuncio de voz al finalizar.

2.5 Herramienta de Gestión de Paquetes

Se soportan dos métodos de instalación:

- **pip**: gestor de paquetes estándar de Python.
- **uv**: gestor moderno y rápido (recomendado). Ver <https://docs.astral.sh/uv/>.

3 Instalación Paso a Paso

3.1 Método 1: Con pip (estándar)

3.1.1 Paso 1 — Clonar el repositorio

```
git clone https://github.com/SebastianG210/proyectoAlgoritmos.git
cd proyectoAlgoritmos
```

Listing 2: Clonar el repositorio desde GitHub.

3.1.2 Paso 2 — Crear un entorno virtual

```
# Crear el entorno
python -m venv .venv

# Activar en Windows PowerShell
.venv\Scripts\Activate.ps1

# Activar en Linux/macOS
source .venv/bin/activate
```

Listing 3: Crear y activar el entorno virtual.

Aviso

En Windows, si aparece el error «*la ejecución de scripts está deshabilitada*», ejecute primero:
`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

3.1.3 Paso 3 — Instalar dependencias

```
pip install -e .
```

Listing 4: Instalar el proyecto en modo editable.

3.1.4 Paso 4 — Verificar la instalación

```
python -c "import numpy, plotly, pandas; print('OK')"
```

Listing 5: Verificar que las dependencias están instaladas correctamente.

Si el resultado es OK, la instalación fue exitosa.

3.2 Método 2: Con uv (recomendado)

```
# Instalar uv (si no est disponible)
pip install uv

# Clonar e instalar en un solo paso
```

```
git clone https://github.com/SebastianG210/proyectoAlgoritmos.git
cd proyectoAlgoritmos
uv sync
```

Listing 6: Instalación rápida con uv.

Consejo

uv sync lee el archivo `pyproject.toml` y crea automáticamente el entorno virtual con todas las dependencias, sin pasos adicionales.

3.3 Estructura de Archivos tras la Instalación

```
1 proyectoAlgoritmos/
2 +-- src/
3 |   +-- shared/
4 |   |   +-- input/      <- Archivos CSV de redes de prueba
5 |   |   \-- output/    <- Resultados generados aqui
6 |   \-- strategies/    <- Algoritmos KGeoMIP y KQNodes
7 +-- pyproject.toml      <- Configuración de dependencias
8 \-- document/          <- Este manual
```

Listing 7: Estructura relevante del proyecto después de instalar.

3.4 Actualización del Software

```
git pull origin master
# Con pip:
pip install -e . --upgrade
# Con uv:
uv sync
```

Listing 8: Actualizar a la versión más reciente.

4 Guía de Uso Básico

4.1 Tutorial en Video

Video Tutorial

Se ha preparado un tutorial en video que cubre la instalación, configuración y primera ejecución del benchmark. El video está disponible en el enlace que se distribuirá junto con este manual. Se recomienda verlo antes de la primera ejecución.

4.2 Formato del Archivo CSV de Entrada

Cada red de prueba se describe mediante un archivo CSV ubicado en `src/shared/input/`.

```
1 state,n0,n1,n2
2 0,0.8,0.3,0.6
3 1,0.2,0.7,0.4
4 2,0.5,0.5,0.9
5 3,0.9,0.1,0.3
6 ...
```

Listing 9: Ejemplo de estructura CSV para una red de 3 nodos.

- **Columna state:** índice del estado presente (entero de 0 a $2^n - 1$).
- **Columnas n0, n1, ...:** probabilidad de que el nodo i tome el valor 1 en el estado siguiente, dado el estado presente.
- El número de filas debe ser exactamente 2^n .

Importante

Si el número de filas no es una potencia de dos exacta, el sistema lanzará un error al construir el N-Cubo. Revise el conteo antes de ejecutar.

4.3 Ejecutar el Benchmark Completo

El script `benchmark.py` ejecuta automáticamente todas las estrategias sobre todas las redes configuradas:

```
cd proyectoAlgoritmos
python src/shared/run/benchmark.py
```

Listing 10: Ejecutar el benchmark completo.

El benchmark muestra el progreso en la terminal con colores:

- Verde: estrategia ejecutada exitosamente.
- Amarillo: modo heurístico activado (umbral superado).
- Rojo: error en la ejecución (ver Sección ??).

4.4 Archivos de Salida CSV

Los resultados se guardan en `src/shared/output/{RED}/{ESTRATEGIA}.csv`.

```
1 index,loss,time,partition
2 0,0.2140,0.0082,"[(1,0),(0,0)],[(1,1),(1,2),(0,1),(0,2)]"
3 1,0.1180,0.0211,"[(1,0)],[(1,1)],[(1,2),(0,0),(0,1),(0,2)]"
```

Listing 11: Ejemplo de archivo de salida generado.

- **index:** número de instancia del benchmark.
- **loss:** pérdida EMD de la partición encontrada (a menor valor, mejor).
- **time:** tiempo de ejecución en segundos.
- **partition:** lista de grupos en formato Python. Cada tupla (`tiempo`, `índice`) representa un vértice donde `tiempo=1` es EFECTO (futuro) y `tiempo=0` es ACTUAL (presente).

4.5 Generar el Dashboard

```
python src/shared/run/dashboard.py
```

Listing 12: Generar el dashboard interactivo.

Se generará el archivo `dashboard.html` en el directorio raíz. Ábralo con cualquier navegador moderno (Chrome, Firefox, Edge).

Consejo

Use `Ctrl + Clic` en los nombres de series dentro del dashboard para mostrar u ocultar estrategias individuales y facilitar la comparación.

4.6 Interpretar la Pérdida EMD

La columna `loss` representa cuánta información se pierde al tratar los grupos de la partición como independientes:

- **0.0:** sin pérdida — las partes son completamente independientes.
- **> 0.3:** alta integración — el sistema tiene fuerte dependencia entre variables.
- La partición con **menor loss** es la *k*-MIP: la más informativa.

5 Parámetros Avanzados

5.1 Parámetro k : Número de Grupos

El parámetro k controla cuántos grupos tendrá la partición:

Cuadro 5: Efecto del parámetro k sobre los resultados.

k	Significado	Pérdida típica
2	Dos grupos (bi-partición)	Mayor
3	Tres grupos	Intermedia
4	Cuatro grupos	Menor
5	Cinco grupos	Mínima

Para ejecutar solo una estrategia con un k específico, modifique `benchmark.py` o llame directamente a la estrategia:

```

1 from src.strategies.GeoMIP.kgeomip import KGeoMIP
2 from src.models.core.system import System
3
4 # Construir el sistema desde CSV
5 sistema = System.desde_csv("src/shared/input/N5B.csv")
6
7 # Preparar subsistema y ejecutar k=3
8 estrategia = KGeoMIP(sistema)
9 resultado = estrategia.ejecutar_k(k=3)
10
11 print(f"Perdida: {resultado.perdida:.4f}")
12 print(f"Particion: {resultado.particion}")

```

Listing 13: Uso directo de KGeoMIP con $k=3$.

5.2 Presupuesto: Umbral Exacto/Heurístico

El parámetro `presupuesto` (por defecto 10 000) determina cuándo se activa el modo heurístico:

- Si el número de k -particiones posibles ($S(n, k)$) es $\leq$ presupuesto: modo **exacto**.
- Si $S(n, k) >$ presupuesto: modo **heurístico** (más rápido, no garantiza óptimo).

```

1 # Forzar modo exacto con presupuesto mayor
2 estrategia = KQNodes(sistema, k=3, presupuesto=100_000)

```

Listing 14: Cambiar el presupuesto para forzar modo exacto.

Aviso

Aumentar el presupuesto puede causar tiempos de ejecución muy largos para redes medianas ($n \geq 15$) con $k \geq 3$. Consulte la Tabla de Números de Stirling en el Apéndice del Manual Técnico antes de modificar este valor.

5.3 Modo Exacto vs. Heurístico

Cuadro 6: Comparación entre modos de búsqueda.

Modo	Garantía	Velocidad	Recomendado para
Exacto	Óptimo global	Lento (exponencial)	$n \leq 10, k \leq 3$
Heurístico	Aproximado	Rápido (polinomial)	$n \geq 15$ o $k \geq 4$

5.4 Selección de Estrategia

Para ejecutar solo KQNodes y no KGeoMIP, edite la lista de estrategias en `benchmark.py`:

```

1 # En benchmark.py, modifique la lista ESTRATEGIAS:
2 ESTRATEGIAS = [
3     ("KQNodes_k2", KQNodes, {"k": 2}),
4     ("KQNodes_k3", KQNodes, {"k": 3}),
5     ("KQNodes_k4", KQNodes, {"k": 4}),
6     ("KQNodes_k5", KQNodes, {"k": 5}),
7 ]

```

Listing 15: Ejecutar solo KQNodes en benchmark.

5.5 Formatos de Salida Disponibles

Actualmente el sistema produce:

- **CSV:** `src/shared/output/{RED}/{ESTRATEGIA}.csv` — un archivo por estrategia.
- **HTML:** `dashboard.html` — dashboard interactivo Plotly.
- **Terminal:** resumen en color con tiempos y pérdidas al finalizar cada ejecución.

Consejo

Para análisis posteriores en Python, cargue los CSV con pandas:

```

1 import pandas as pd
2 df = pd.read_csv("src/shared/output/N10A/KGeoMIP_k3.csv")
3 print(df.describe())

```

6 Solución de Problemas

6.1 Errores de Instalación

6.1.1 Error: `ModuleNotFoundError: No module named 'numpy'`

Causa: Las dependencias no fueron instaladas o el entorno virtual no está activado.

```
# Verificar que el entorno virtual esta activo
# En Windows:
.venv\Scripts\Activate.ps1
# En Linux/macOS:
source .venv/bin/activate

# Reinstalar
pip install -e .
```

Listing 16: Solución: reinstalar dependencias.

6.1.2 Error: `pyttsx3` no funciona en Linux/macOS

Causa: `pyttsx3` requiere un motor de síntesis de voz del sistema operativo.

```
sudo apt-get install espeak
pip install pyttsx3
```

Listing 17: Instalar dependencia de voz en Ubuntu.

En macOS, el motor *NSSpeechSynthesizer* debería funcionar sin instalación adicional. Si el error persiste, ignore la síntesis de voz: el benchmark funciona normalmente sin ella.

6.2 Errores de Formato en CSV

6.2.1 Error: `ValueError: El tamaño del CSV no es potencia de 2`

Causa: El número de filas no corresponde a 2^n para ningún valor de n .

Solución: Abra el CSV y verifique que tenga exactamente 2^n filas de datos (sin contar el encabezado). Para $n = 3$ deben ser 8 filas, para $n = 5$ son 32 filas, etc.

6.2.2 Error: `KeyError: 'state'`

Causa: El encabezado del CSV no contiene la columna `state` (con esa ortografía exacta).

Solución: Asegúrese de que la primera columna del CSV se llama `state` (en minúsculas).

6.3 Errores de Memoria

6.3.1 Error: `MemoryError` al ejecutar redes grandes

Causa: El modo exacto para $k \geq 3$ y $n \geq 15$ puede requerir varios GB de RAM.

```

1 # Forzar modo heurístico con presupuesto muy bajo
2 estrategia = KQNodes(sistema, k=3, presupuesto=1_000)

```

Listing 18: Reducir el presupuesto para forzar modo heurístico.

Aviso

El modo heurístico con presupuesto muy bajo puede dar resultados de baja calidad. Balance entre memoria disponible y calidad del resultado.

6.4 Errores de Ejecución del Benchmark

6.4.1 El benchmark termina con pérdida = 0 para todos los casos

Causa posible A: El subsistema analizado tiene un solo nodo (no se puede particionar).

Causa posible B: El estado inicial coincide con todos los nodos en el mismo estado.

Solución: Verifique que el archivo CSV tiene variabilidad en las probabilidades de transición.

6.4.2 El benchmark no genera archivos CSV en output/

Causa: Los directorios de salida no existen.

```

# En Windows PowerShell:
New-Item -ItemType Directory -Force src\shared\output\N10A
# En Linux/macOS:
mkdir -p src/shared/output/N10A

```

Listing 19: Crear directorios de salida manualmente.

6.5 Tabla Resumen de Errores Comunes

Error	Causa	Solución
ModuleNotFoundError	Entorno no activado	Activar venv y reinstalar
ValueError: not power of 2	CSV con filas incorrectas	Verificar 2 ⁿ filas
KeyError: 'state'	Encabezado incorrecto	Renombrar columna
MemoryError	Redes grandes modo exacto	Reducir presupuesto
Pérdida siempre 0	Sistema sin variabilidad	Revisar CSV de entrada
Sin archivos en output/	Directorios no existen	Crear con mkdir
Tiempo excesivo (>10 min)	Modo exacto con $k \geq 4$	Reducir k o presupuesto

7 Ejemplos y Tutoriales

7.1 Ejemplo 1: Red N5B con k=3 (Básico)

Este tutorial muestra cómo analizar la red de 5 nodos N5B con una 3-partición.

Paso 1 — Verificar el archivo de entrada:

```
# Verificar que el CSV existe
ls src/shared/input/N5B.csv
# Debe mostrar el archivo (32 filas de datos + encabezado)
```

Paso 2 — Ejecutar KGeoMIP con k=3:

```
1 # Desde la raiz del proyecto:
2 from src.strategies.GeoMIP.kgeomip import KGeoMIP
3 from src.models.core.system import System
4
5 sistema = System.desde_csv("src/shared/input/N5B.csv")
6 kgeo = KGeoMIP(sistema)
7 resultado = kgeo.ejecutar_k(k=3)
8
9 print(f"Perdida (EMD): {resultado.perdida:.4f}")
10 print(f"Particion: {resultado.particion}")
11 print(f"Tiempo: {resultado.tiempo:.3f} s")
```

Listing 20: Ejecutar KGeoMIP sobre N5B con 3 grupos.

Salida esperada:

```
1 Perdida (EMD): 0.1180
2 Particion: [[(1,0)], [(1,1), (0,0)], [(1,2), (0,1), (0,2)]]
3 Tiempo: 0.021 s
```

Listing 21: Salida del ejemplo 1.

Interpretación: La 3-partición divide los 5 nodos en tres grupos. La pérdida de 0.118 es aproximadamente la mitad que la bi-partición (0.214), lo que indica que los tres grupos explican mejor la dinámica del sistema de forma independiente.

7.2 Ejemplo 2: Red N10A con k=4 (Intermedio)

Paso 1 — Ejecutar el benchmark selectivo:

```
1 import time
2 from src.strategies.QNodes.kqnodes import KQNodes
3 from src.models.core.system import System
4
5 sistema = System.desde_csv("src/shared/input/N10A.csv")
6 kqn = KQNodes(sistema, k=4)
7
8 inicio = time.time()
9 resultado = kqn.resolver()
10 fin = time.time()
11
```

```

12 print(f"Modo: {'exacto' if resultado.modo == 'exacto' else 'greedy'}")
13 print(f"Perdida: {resultado.perdida:.4f}")
14 print(f"Tiempo total: {fin - inicio:.2f} s")

```

Listing 22: Benchmark selectivo para N10A.

Resultado esperado: El sistema reportará modo *greedy* (porque $S(10,4) = 34,105 > 10,000$) con una pérdida aproximada de 0.148.

7.3 Ejemplo 3: Comparar GeoMIP vs KGeoMIP

```

1 from src.strategies.GeoMIP.geomip import GeoMIP
2 from src.strategies.GeoMIP.kgeomip import KGeoMIP
3 from src.models.core.system import System
4
5 sistema = System.desde_csv("src/shared/input/N5B.csv")
6
7 geo = GeoMIP(sistema)
8 kgeo = KGeoMIP(sistema)
9
10 r_geo = geo.resolver()
11 r_kgeo = kgeo.ejecutar_k(k=2)
12
13 dif = abs(r_geo.perdida - r_kgeo.perdida)
14 print(f"GeoMIP loss: {r_geo.perdida:.6f}")
15 print(f"KGeoMIP loss: {r_kgeo.perdida:.6f}")
16 print(f"Diferencia: {dif:.2e} ({'OK' if dif < 1e-5 else 'FALLO'})")

```

Listing 23: Comparación directa entre GeoMIP y KGeoMIP(k=2).

La diferencia debe ser menor que 10^{-5} , validando TV-02.

7.4 Ejemplo 4: Ejecutar el Benchmark Completo y Generar Dashboard

```

# Ejecutar todas las estrategias
python src/shared/run/benchmark.py

# Generar el dashboard interactivo
python src/shared/run/dashboard.py

# Abrir el dashboard (Windows)
start dashboard.html

# Abrir el dashboard (Linux)
xdg-open dashboard.html

# Abrir el dashboard (macOS)
open dashboard.html

```

Listing 24: Flujo completo: benchmark + dashboard.

Consejo

El dashboard permite hacer zoom, exportar gráficas como PNG y comparar estrategias de forma interactiva. Use la barra de herramientas de Plotly en la esquina superior derecha de cada gráfica.

8 Referencia Rápida

8.1 Tabla de Parámetros

Cuadro 8: Parámetros de las estrategias KGeoMIP y KQNodes.

Parámetro	Tipo	Por defecto	Rango	Descripción
k	int	2	2...5	Número de grupos
presupuesto	int	10 000	1...∞	Umbral exacto/heurístico
semilla	int	42	cualquiera	Semilla NumPy
modo_emd	str	'l1'	'l1', 'emd'	Métrica de distancia

8.2 Comandos Esenciales

Cuadro 9: Comandos de terminal más usados.

Comando	Acción
<code>python src/shared/run/benchmark.py</code>	Ejecutar benchmark completo
<code>python src/shared/run/dashboard.py</code>	Generar dashboard HTML
<code>pip install -e .</code>	Instalar/actualizar dependencias
<code>uv sync</code>	Sincronizar entorno con uv
<code>python -c "import numpy; print(numpy.__version__)"</code>	Verificar numpy

8.3 Archivos y Rutas Clave

Cuadro 10: Rutas importantes del proyecto.

Ruta	Contenido
<code>src/shared/input/*.csv</code>	Archivos de red de entrada
<code>src/shared/output/{RED}/{EST}.csv</code>	Resultados del benchmark
<code>dashboard.html</code>	Dashboard interactivo
<code>src/strategies/GeoMIP/kgeomip.py</code>	Código KGeoMIP
<code>src/strategies/QNodes/kqnodes.py</code>	Código KQNodes
<code>pyproject.toml</code>	Dependencias del proyecto

8.4 Glosario

k-MIP	<i>k-Minimum Information Partition</i> . La partición de k grupos que minimiza la pérdida de información EMD.
EMD	<i>Earth Mover's Distance</i> . Medida de cuán diferentes son dos distribuciones de probabilidad. En este proyecto, cuantifica la pérdida al separar el sistema en grupos.
IIT	<i>Teoría de la Información Integrada</i> . Marco matemático para cuantificar la conciencia de un sistema mediante la métrica Φ (phi).

N-Cubo	Representación hipercúbica de la dinámica de un nodo binario. Con n nodos, el espacio de estados tiene 2^n puntos.
TPM	<i>Transition Probability Matrix</i> . Tabla que describe con qué probabilidad el sistema pasa de cada estado presente a cada estado futuro.
Stirling $S(n, k)$	Número de formas de dividir n elementos en exactamente k grupos. Crece exponencialmente: $S(10, 4) \approx 34,000$.
Modo exacto	El algoritmo evalúa <i>todas</i> las $S(n, k)$ particiones posibles. Garantiza el óptimo pero puede ser lento.
Modo heurístico	El algoritmo usa atajos geométricos (KGeoMIP) o greedy MAO (KQNodes) para explorar solo una fracción de las particiones. Más rápido, resultado aproximado.
MAO	<i>Maximum Adjacency Ordering</i> . Algoritmo greedy que construye la partición añadiendo iterativamente el elemento más conectado al grupo actual.
Oráculo Zeta	Estructura precalculada que permite consultar la función submodular de QNodes en tiempo constante $O(1)$ por consulta.
GeoMIP	Estrategia base de bi-partición basada en la geometría del hipercubo n -dimensional y la tabla de costos Hamming.
KGeoMIP	Extensión de GeoMIP a k grupos ($k = 2 \dots 5$).
QNodes	Estrategia base de bi-partición basada en el algoritmo de Queyranne para funciones submodulares, con precómputo de la transformada Zeta.
KQNodes	Extensión de QNodes a k grupos ($k = 2 \dots 5$).
Pérdida (loss)	Valor EMD de la mejor partición encontrada. Un valor cercano a 0 indica que el sistema puede descomponerse en partes casi independientes.

8.5 Contacto y Soporte

Para reportar errores o solicitar nuevas funcionalidades, abra un *issue* en el repositorio de GitHub del proyecto:

<https://github.com/SebastianG210/proyectoAlgoritmos/issues>

Nota

Incluya en el reporte: versión de Python, sistema operativo, el CSV de entrada, el comando ejecutado y el mensaje de error completo.